

Report 4: Commander Module

Part I: Complete Original System — Every Function

Part II: Every Soaring-Mode Modification and Its Impact

PX4 Autosoaring Implementation — Defence Technical Report

Radhouene — Fixed-Wing Autosoaring Project

April 2026

Contents

List of Symbols and Acronyms	3
I Original Commander System	5
1 Module Purpose and Architecture	5
2 task_spawn()	6
3 run() — The Main Control Loop	6
3.1 Initialization (once before the loop)	6
3.2 Per-Cycle Steps (executed every iteration)	6
4 Navigator → Commander Data Flow	9
4.1 Channel 1: vehicle_command (active commands)	10
4.2 Channel 2: mission_result (mission state feedback)	10
4.3 Channel 3: geofence_result (fence violation)	10
4.4 Channel 4: navigator_status (Navigator health)	11
4.5 Summary table	11
5 handle_command() — All Command Types	11
5.1 VEHICLE_CMD_DO_SET_MODE (176)	11
5.2 VEHICLE_CMD_COMPONENT_ARM_DISARM (400)	12
5.3 VEHICLE_CMD_DO_REPOSITION (192)	12
5.4 VEHICLE_CMD_DO_CHANGE_ALTITUDE (186)	12
5.5 VEHICLE_CMD_PREFLIGHT_CALIBRATION (241)	13
5.6 VEHICLE_CMD_RUN_PREARM_CHECKS (401)	13
5.7 VEHICLE_CMD_PREFLIGHT_REBOOT_SHUTDOWN (246)	13
5.8 Other Commands Processed by handle_command	13

6	custom_command() — All CLI Commands	13
6.1	calibrate	14
6.2	check	14
6.3	arm / disarm	14
6.4	takeoff / land	14
6.5	transition (VTOL)	14
6.6	mode	15
6.7	lockdown	15
6.8	pair	15
6.9	set_ekf_origin <lat> <lon> <alt>	15
6.10	poweroff	15
7	executeActionRequest()	15
8	handleModeIntentionAndFailsafe()	16
9	updateControlMode()	16
10	control_status_leds()	17
11	dataLinkCheck(), manualControlCheck(), battery_status_check()	17
11.1	dataLinkCheck()	17
11.2	manualControlCheck()	17
11.3	battery_status_check()	17
12	Other Functions	18
II	Soaring-Mode Modifications	18
13	Overview of Changes	18
14	Modification C1: New CLI Soaring Commands	18
14.1	What Was Added	18
14.2	Two-Command Pattern	19
14.3	VEHICLE_CMD_CUSTOM_0 Processing (in fw_pos_control)	19
14.4	Override Clearing Conditions	20
15	Modification C2: DO_REPOSITION param2=0 Pass-Through	20
15.1	Original Behaviour	20
15.2	Modified Behaviour	20
15.3	Impact on Non-Soaring Flight	20
16	Summary Table: All Modifications to Commander	21

List of Symbols and Acronyms

Abbreviations

Symbol	Meaning
uORB	Micro Object Request Broker — PX4's publish/subscribe IPC bus
MAVLink	Micro Air Vehicle Link — binary telemetry/command protocol
DDS	Data Distribution Service — middleware used by XRCE-DDS for ROS2
CLI	Command-Line Interface
SITL	Software-In-The-Loop simulation
HIL	Hardware-In-The-Loop simulation
RTL	Return-To-Launch
GCS	Ground Control Station (e.g. QGroundControl)
VTOL	Vertical Take-Off and Landing
LED	Light-Emitting Diode (status indicator)

Key uORB Topics Published / Subscribed by Commander

Topic	Direction	Content
vehicle_status	Published	Navigation state, arming state, failsafe flags
vehicle_control_mode	Published	Boolean control-mode flags (altitude, position, etc.)
actuator_armed	Published	Armed, prearmed, lockdown, force_failsafe
failure_detector_status	Published	Failure detector roll/pitch/alt/motor flags
vehicle_command	Subscribed	Incoming MAVLink-format commands
vehicle_command_ack	Published	Command acknowledgement result codes
action_request	Subscribed	Arm/disarm/mode-switch requests from RC/joystick
parameter_update	Subscribed	Signals parameter changes
battery_status	Subscribed	Battery SOC/voltage for failsafe thresholds
manual_control_input	Subscribed	Pilot stick inputs
mission_result	Subscribed	Mission finished/valid flags
vehicle_land_detected	Subscribed	Landed / in-air state
power_button_state	Subscribed	Physical power button events
datalink_status	Subscribed	Telemetry link health
offboard_control_mode	Subscribed	Offboard mode setpoint validity

Navigation State Constants (nav_state field of vehicle_status)

Constant	Flight mode
NAVIGATION_STATE_MANUAL	Manual (no autopilot)
NAVIGATION_STATE_ALTCTL	Altitude control
NAVIGATION_STATE_POSCTL	Position control
NAVIGATION_STATE_AUTO_MISSION	Follow uploaded mission
NAVIGATION_STATE_AUTO_LOITER	Circle a fixed point
NAVIGATION_STATE_AUTO_RTL	Return to launch
NAVIGATION_STATE_AUTO_TAKEOFF	Automatic takeoff
NAVIGATION_STATE_AUTO_LAND	Automatic landing
NAVIGATION_STATE_AUTO_PRECLAND	Precision landing
NAVIGATION_STATE_ACRO	Acrobatic / rate control
NAVIGATION_STATE_STAB	Stabilised (roll/pitch damped)
NAVIGATION_STATE_OFFBOARD	External position/velocity setpoints
NAVIGATION_STATE_TERMINATION	Kill switch / termination
NAVIGATION_STATE_ORBIT	GCS-commanded orbit

MAVLink Command IDs used in Commander

Constant	Purpose
VEHICLE_CMD_DO_SET_MODE (176)	Switch flight mode
VEHICLE_CMD_COMPONENT_ARM_DISARM (400)	Arm or disarm motors
VEHICLE_CMD_DO_REPOSITION (192)	Move loiter centre
VEHICLE_CMD_PREFLIGHT_CALIBRATION (241)	Sensor calibration
VEHICLE_CMD_RUN_PREARM_CHECKS (401)	Run safety checks
VEHICLE_CMD_PREFLIGHT_REBOOT_SHUTDOWN (246)	Reboot / shutdown FMU
VEHICLE_CMD_NAV_TAKEOFF (22)	Auto takeoff
VEHICLE_CMD_NAV_LAND (21)	Auto land
VEHICLE_CMD_DO_FLIGHTTERMINATION (185)	Lockdown / kill
VEHICLE_CMD_DO_VTOL_TRANSITION (3000)	MC↔FW transition
VEHICLE_CMD_CUSTOM_0 (31010)	User-defined; soaring mode here
VEHICLE_CMD_FIXED_MAG_CAL_YAW (42006)	Fast magnetometer calibration

Part I

Original Commander System

1 Module Purpose and Architecture

Commander is the *system state manager* and *safety arbiter* of PX4. It is the single authoritative source of the vehicle's **arming state** and **navigation state**. Every other module reads these from the `vehicle_status` uORB topic; none of them switch states themselves.

Commander has no direct link to sensors or actuators. Its data flow is:

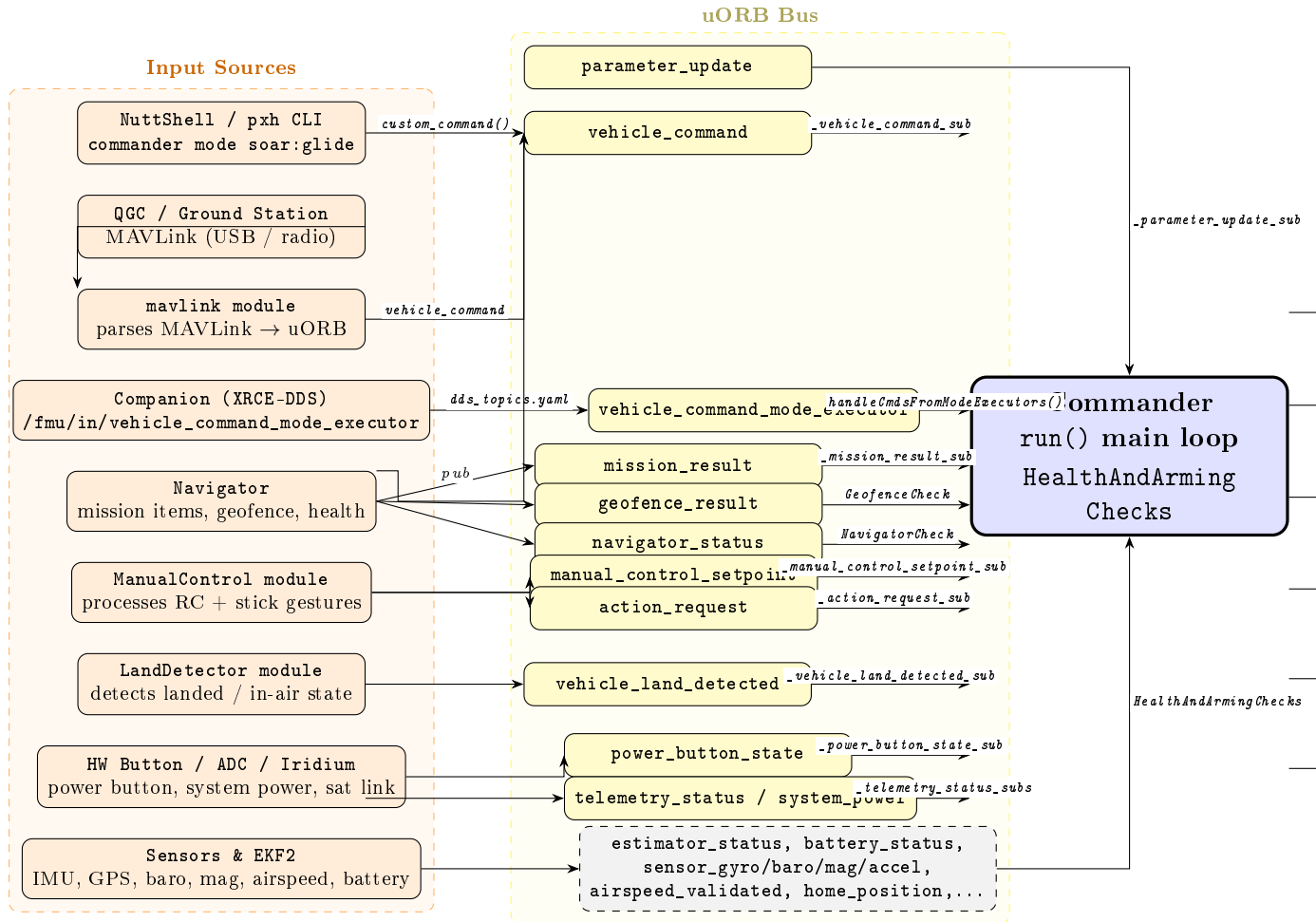


Figure 1: Commander complete input/output data-flow (verified from source code). All input channels are uORB topics; Commander never reads sensors or actuators directly. The companion computer has *two* separate paths: MAVLink (via the `mavlink module` → `vehicle_command`) and XRCE-DDS (`vehicle_command_mode_executor`). The `action_request` topic is published *exclusively* by the `ManualControl` module from RC pilot inputs, never by the companion.

Complete verified input channel list

The following table lists every uORB topic that Commander (including its `HealthAndArmingChecks` subsystem) subscribes to, verified directly from the source code in `Commander.hpp` and `HealthAndArmingChecks/checks/*.hpp`.

uORB topic	Publisher module	Role in Commander
<code>vehicle_command</code>	CLI, mavlink, Navigator, ManualControl, fw_pos_control, RC drivers, ...	Main command dispatch (<code>handle_command()</code>)
<code>vehicle_command_mode_executor</code>	Companion via XRCE-DDS	Authenticated mode-executor commands (<code>handleCommandsFromModeExecutors()</code>)
<code>mission_result</code>	Navigator	Mission valid/finished → arming check and auto-loiter on finish
<code>geofence_result</code>	Navigator	Fence breach → failsafe action (<code>GeofenceCheck</code>)
<code>navigator_status</code>	Navigator	Navigator failure / HAGL violation (<code>NavigatorCheck</code>)
<code>manual_control_setpoint</code>	ManualControl module	RC stick gestures and mode slot selection
<code>action_request</code>	ManualControl module <i>only</i> (RC)	Arm/disarm/kill/mode from pilot switches and stick gestures
<code>vehicle_land_detected</code>	LandDetector module	In-air vs. landed state for failsafe decisions
<code>power_button_state</code>	Commander itself (HW interrupt callback)	Power-button long-press → shutdown command
<code>telemetry_status</code>	mavlink module (one per instance)	RC/data-link health for link-loss failsafe
<code>system_power</code>	ADC driver / power simulator	USB connected, brick valid, servo power state
<code>iridiumsbd_status</code>	IridiumSBD driver	Satellite link health (optional, failsafe aware)
<code>offboard_control_mode</code>	mavlink_receiver (MAVLink SET_POSITION_TARGET_*)	Offboard mode available check
<code>vtol_vehicle_status</code>	vtol_att_control	VTOL transition state, fixed-wing system failure
<code>parameter_update</code>	param subsystem	Trigger parameter reload (1 Hz interval)
<code>cpuload</code>	LoadMon module	CPU/RAM overload arming check
<code>estimator_status</code> / <code>_flags</code> / <code>_sensor_bias</code>	EKF2	EKF health, innovations, bias (<code>EstimatorCheck</code>)
<code>vehicle_global/local_position</code>	EKF2	Position validity for mode gating
<code>vehicle_attitude</code> , <code>vehicle_angular_velocity</code>	EKF2 / attitude estimator	Attitude health check
<code>vehicle_gps_position</code>	GPS driver / EKF2	GPS fix quality check
<code>battery_status</code> (multi)	Battery estimator	Battery failsafe level, RTL time estimate
<code>sensor_gyro/baro/mag/accel</code> (multi)	IMU/baro/mag/accel drivers	Sensor redundancy health checks
<code>airspeed_validated</code>	Airspeed estimator	Airspeed sensor health (<code>AirspeedCheck</code>)
<code>manual_control_switches</code>	ManualControl module	Switch calibration check (<code>ManualControlCheck</code>)
<code>home_position</code>	home position estimator	Home set check (<code>HomePositionCheck</code>)
<code>esc_status</code>	ESC driver	ESC health check
<code>wind</code>	wind estimator	Wind estimate health
<code>logger_status</code>	logger module	Logging running check
<code>distance_sensor</code> (multi)	range finder driver	Distance sensor health
<code>arming_check_reply</code>	external mode executors	External pre-arm check results

Key design principle: Commander acts on *requests* (vehicle commands, action requests, manual stick inputs), validates them against preconditions (arming, preflight checks, failsafe state), and only then changes the system state. It never reads sensors directly — every input, from GPS fix quality to battery voltage, arrives through a uORB topic published by a dedicated driver or estimator.

2 task_spawn()

```
1 int Commander::task_spawn(int argc, char *argv[])
```

Called by the PX4 module framework when the user runs `commander start`.

1. Creates the Commander task with `px4_task_spawn_cmd`, priority `SCHED_PRIORITY_DEFAULT+40`, stack 3400 bytes.
2. Returns the task handle.

3 run() — The Main Control Loop

The main loop runs indefinitely at approximately 10–100 Hz (event-driven by uORB updates).

3.1 Initialization (once before the loop)

1. `led_init()` and `buzzer_init()` — initialise RGB LED and piezo buzzer hardware drivers.
2. On boards with `BOARD_HAS_POWER_CONTROL`: pre-allocate `power_button_state` and `tune_control` uORB buffers (uORB cannot allocate in IRQ context).
3. Register `power_button_state_notification_cb` as the power button ISR callback (so button events arrive immediately).
4. Record `_boot_timestamp = hrt_absolute_time()`.
5. `arm_auth_init()` — initialise the arm-authorisation subsystem (used by some integrators for external arm approval).

3.2 Per-Cycle Steps (executed every iteration)

Each loop iteration performs the following steps in order:

1. Parameter update. If `_parameter_update_sub.updated()`: copy the update, call `updateParameters()`, set `_status_changed = true`.

2. `handlePowerButtonState()`. Reads `power_button_state`. If the button was held for long enough, initiates a controlled shutdown sequence via `VEHICLE_CMD_PREFLIGHT_REBOOT_SHUTDOWN`.

3. `systemPowerUpdate()`. Reads USB/power connector states. Updates `_vehicle_status.usb_connected` and `_vehicle_status.circuit_breaker_engaged`. Detects power-supply anomalies.

4. `landDetectorUpdate()`. Polls `vehicle_land_detected`. Transitions arming state on landing (auto-disarm on ground after configurable timeout `COM_DISARM_LAND`). Sets `_vehicle_land_detected.landed` for use downstream.

5. `safetyButtonUpdate()`. Reads the hardware safety switch (if present). Arms/disarms when the safety button is pressed (depending on parameter `COM_SAFETY_EN`).

6. `vtolStatusUpdate()`. Polls `vtol_vehicle_status`. Updates `_vehicle_status.vehicle_type` (rotary-wing vs. fixed-wing) and `in_transition_mode` flag during VTOL transition manoeuvres.

7. Home position update. If `COM_HOME_EN` is set and the vehicle is landed: `_home_position.update` attempts to set the home position from the current GPS fix. Sets `home_position` uORB.

8. `handleAutoDisarm()`. Implements several auto-disarm policies:

- Auto-disarm on ground after landing if `COM_DISARM_LAND > 0`.
- Auto-disarm when never-armed timeout expires (`COM_DISARM_PRFLT`).
- Auto-disarm in air if `COM_DISARM_MAN` is set and kill switch activated.

9. `battery_status_check()`. Reads all battery status topics. Computes the worst-case battery warning level (none / low / critical / emergency). Sets failsafe flags if battery is below configured thresholds. Stores the battery warning level in `_battery_warning` for the LED driver.

10. `checkForMissionUpdate()`. Polls `mission_result`. If the mission result changed (new valid mission loaded, mission finished, mission failed), updates internal state and sets `_status_changed`.

11. `manualControlCheck()`. Reads `manual_control_input`.

- Detects RC link loss and sets `_vehicle_status.rc_signal_lost`.
- Maps RC mode-switch channels to `action_request` events (arm, disarm, mode change).
- Implements stick-arming (sticks to corners for 2s).

12. `offboardControlCheck()`. Reads `offboard_control_mode`. If an offboard stream was active but has gone stale (no update within timeout), sets `_vehicle_status.offboard_control_status`.

13. `dataLinkCheck()`. Reads `datalink_status` for all connected telemetry links. Manages the GCS link-loss failsafe. Updates `_vehicle_status.data_link_lost` and triggers the configured failsafe action (warn / RTL / loiter / disarm) after the configured timeout (`COM_DL_LOSS_T`).

14. Failure detector update. `_failure_detector.update()` runs attitude-envelope checks:

- Roll exceedance (`CBRK_FLIGHTTERM`: roll $> 60^\circ$ for > 200 ms triggers flight termination).
- Pitch exceedance.
- Altitude exceedance.

- External failure signal.
- Motor failure (from ESC status).

Updates `_vehicle_status.failure_detector_status`.

15. `modeManagementUpdate()`. `_mode_management.update()` refreshes the set of available navigation states (e.g. removes modes whose executor is not running). If an unavailable mode was user-intended, Commander switches to the nearest valid fallback.

16. `handleModeIntentionAndFailsafe()`. This is the **most critical function** in the run loop. It:

1. Builds the failsafe state struct from all current status flags.
2. Calls `_failsafe.update()` which evaluates all active failsafe triggers in priority order and selects the highest-priority failsafe action (none / warn / RTL / loiter / disarm / terminate).
3. Maps the selected failsafe action to a `nav_state` via `FailsafeBase::modeFromAction()`.
4. Executes the action:
 - `Action::Disarm`: calls `disarm()`.
 - `Action::Terminate`: forces `nav_state = NAVIGATION_STATE_TERMINATION`.
 - All other actions: just set `nav_state`.
5. Updates failsafe defer state flags.
6. Returns `true` if `nav_state` changed (triggers immediate re-publish).

17. Arming checks (at 10 Hz). `_health_and_arming_checks.update()` re-evaluates all preflight conditions (IMU health, GPS fix, calibration status, RC link, etc.). Result stored in `_vehicle_status.pre_flight_checks_pass`. `checkAndInformReadyForTakeoff()` prints a green-text SITL message.

18. `handleCommandsFromModeExecutors()`. Processes arm/disarm requests that come from *mode executors* (e.g. an external mode that needs to force a disarm). These have a dedicated path separate from the main `vehicle_command` topic.

19. Vehicle command dispatch. If `_vehicle_command_sub.updated()`:

1. Copy the command.
2. Call `handle_command(cmd)` (described in the next section).
3. If `handle_command` returns `true`: set `_status_changed`.

20. Action request dispatch. If `_action_request_sub.updated()`:

1. Copy the action request.
2. Call `executeActionRequest(action_request)`.

21. actuator_armed update. Rebuild `actuator_armed_s` from current state and publish if changed:

- `armed = isArmed()`.
- `prearmed = getPrearmState()` (ESCs can be spin-up-tested without full arm).
- `ready_to_arm = pre_flight_checks_pass || armed`.
- `lockdown = HIL mode or throw-launch in progress`.
- `force_failsafe = nav_state == TERMINATION`.
- If `force_failsafe` or `manual_lockdown` newly set while armed: call `send_parachute_command()`

22. Status publish (at 2 Hz or on change). In order:

1. Publish `actuator_armed`.
2. `updateControlMode()` — rebuild and publish `vehicle_control_mode` boolean flags from `nav_state`.
3. Publish `vehicle_status`.
4. Publish `failure_detector_status`.

23. Housekeeping. `checkWorkerThread()`, `updateTunes()`, `control_status_leds()`, `clear_status_changed`, call `arm_auth_update()`, `px4_indicate_external_reset_lockout()`.

4 Navigator → Commander Data Flow

Navigator is the PX4 module that executes the mission plan and manages the geofence. It does *not* call Commander directly; all communication goes through uORB topics. There are **four** distinct channels.

4.1 Channel 1: `vehicle_command` (active commands)

When the Navigator encounters a mission item that requires a mode change (e.g. `MAV_CMD_DO_SET_MODE`, `MAV_CMD_MISSION_START`, `MAV_CMD_DO_REPOSITION`), it calls its internal `publish_vehicle_command()` helper which publishes to the `vehicle_command` uORB topic. Commander's main loop polls this topic via `_vehicle_command_sub` and processes it in `handle_command()`.

Navigator publishes	Commander action
<code>DO_SET_MODE</code> (e.g. <code>AUTO_MISSION</code>)	switches <code>nav_state</code>
<code>MISSION_START</code>	triggers takeoff sequence
<code>DO_REPOSITION</code>	thermal-centering loiter radius

4.2 Channel 2: mission_result (mission state feedback)

Navigator publishes `mission_result_s` whenever:

- A new mission is uploaded (new `mission_id`).
- The mission becomes valid or invalid.
- The mission finishes (last waypoint reached).
- A warning is detected (e.g. waypoint below terrain).

Commander subscribes via `_mission_result_sub` (`SubscriptionData<mission_result_s>` in `Commander.hpp`). It uses the data to:

- Block arming if the mission is invalid.
- Play a tone when a valid mission is loaded.
- Transition to `AUTO_LOITER` when `mission_result.finished == true` and the vehicle is still airborne (“mission complete” loiter).
- Check `seq_total` to validate `DO_JUMP` targets.

4.3 Channel 3: geofence_result (fence violation)

Navigator evaluates the geofence every cycle and publishes `geofence_result_s` containing boolean flags: `geofence_max_dist_triggered`, `geofence_max_alt_triggered`, `geofence_custom_fence_triggered`, and the configured `geofence_action` (warn / hold / RTL / terminate).

Commander’s `HealthAndArmingChecks` subsystem includes `GeofenceChecks` (in `HealthAndArmingChecks`). It subscribes via `_geofence_result_sub` and:

1. Sets a failsafe flag if any fence is triggered.
2. Maps the action to an arming-check failure or failsafe escalation (e.g. `GF_ACTION_RTL` → forces RTL mode).

This is the path by which a geofence violation becomes a failsafe action inside Commander, without Commander needing to know any fence coordinates.

4.4 Channel 4: navigator_status (Navigator health)

Navigator publishes `navigator_status_s` containing a `failure` field (enum: `FAILURE_NONE`, `FAILURE_HAGL`, etc.). Commander’s `NavigatorChecks` (`HealthAndArmingChecks/checks/navigatorChecks`) subscribes via `_navigator_status_sub` and sets:

```
reporter.failsafeFlags().navigator_failure =  
    (status.failure != navigator_status_s::FAILURE_NONE);
```

If the current `nav_state` matches the reported state and the failure is `FAILURE_HAGL` (height-above-ground-level constraint violated), Commander logs an error and blocks the mode.

4.5 Summary table

uORB topic	Publisher	Subscriber in Commander	Purpose
vehicle_command	Navigator	_vehicle_command_sub	Mode changes from mission items
mission_result	Navigator	_mission_result_sub	Mission valid/finished/warning
geofence_result	Navigator	GeofenceChecks	Fence violation → failsafe
navigator_status	Navigator	NavigatorChecks	Navigator health / HAGL failure

Important: Commander does *not* receive individual waypoint coordinates from the mission. Those go from Navigator directly to `fw_pos_control` via `position_setpoint_triplet`, completely bypassing Commander. Commander only learns the mission outcome (`mission_result`) or receives explicit mode-change commands (`vehicle_command`).

5 handle_command() — All Command Types

```
1 bool Commander::handle_command(const vehicle_command_s &cmd)
```

First checks that the command is addressed to this system/component (or is broadcast with `target = 0`). If not, returns false immediately.

Target routing:

$$\text{route} = ((\text{cmd.target_system} == \text{system_id}) \vee (\text{cmd.target_system} == 0)) \wedge ((\text{cmd.target_component} == \text{COMP_NAV}) \vee (\text{cmd.target_component} == 0)) \quad (1)$$

5.1 VEHICLE_CMD_DO_SET_MODE (176)

Decodes `param1` (base mode flags), `param2` (custom main mode), `param3` (custom sub mode):

custom_main_mode	custom_sub_mode	nav_state
MANUAL	—	NAVIGATION_STATE_MANUAL
ALTCTL	—	NAVIGATION_STATE_ALTCTL
POSCTL	0	NAVIGATION_STATE_POSCTL
POSCTL	SLOW	NAVIGATION_STATE_POSITION_SLOW
AUTO	LOITER	NAVIGATION_STATE_AUTO_LOITER
AUTO	MISSION	NAVIGATION_STATE_AUTO_MISSION
AUTO	RTL	NAVIGATION_STATE_AUTO_RTL
AUTO	TAKEOFF	NAVIGATION_STATE_AUTO_TAKEOFF
AUTO	LAND	NAVIGATION_STATE_AUTO_LAND
AUTO	PRECLAND	NAVIGATION_STATE_AUTO_PRECLAND
AUTO	FOLLOW_TARGET	NAVIGATION_STATE_AUTO_FOLLOW_TARGET
AUTO	EXTERNAL1–8	NAVIGATION_STATE_EXTERNAL1 + offset
ACRO	—	NAVIGATION_STATE_ACRO
STABILIZED	—	NAVIGATION_STATE_STAB
OFFBOARD	—	NAVIGATION_STATE_OFFBOARD

If the mode is valid: calls `_user_mode_intention.change(desired_nav_state)`. If the mode executor is not running or the vehicle cannot enter the mode: answers with `TEMPORARILY_REJECTED`.

5.2 VEHICLE_CMD_COMPONENT_ARM_DISARM (400)

- `param1 = ARMING_ACTION_ARM`: calls `arm()`. Checks: preflight pass, not in HIL, not locked down, arm-auth granted.
- `param1 = ARMING_ACTION_DISARM`: calls `disarm()`. Checks: not in auto-flight (unless forced with `param2 = 21196`).

Force flag (`param2 = 21196`) bypasses in-flight disarm protection.

5.3 VEHICLE_CMD_DO_REPOSITION (192)

```
1 const uint32_t change_mode_flags = uint32_t(cmd.param2);
2 const bool mode_switch_not_requested = (change_mode_flags & 1) == 0;
3 const bool unsupported_bits_set      = (change_mode_flags & ~1) != 0;
```

- If `param2 == 0` (no mode switch): accept immediately with `ACCEPTED`; the navigator will process the coordinates regardless.
- If `param2 == 1` (request mode switch to LOITER): calls `_user_mode_intention.change(NAVIGATOR_MODE_LOITER)`.
- If unsupported bits set: reject with `UNSUPPORTED`.

5.4 VEHICLE_CMD_DO_CHANGE_ALTITUDE (186)

Switches mode to LOITER. Coordinates are processed by the navigator.

5.5 VEHICLE_CMD_PREFLIGHT_CALIBRATION (241)

Routes to the calibration worker thread based on which param is non-zero:

param	Calibration
<code>param1 = 1</code>	Gyroscope
<code>param2 = 1</code>	Magnetometer
<code>param3 = 1</code>	Barometer
<code>param5 = 1</code>	Accelerometer (full)
<code>param5 = 3</code>	Accelerometer (quick)
<code>param5 = 2</code>	Level (board tilt)
<code>param6 = 2</code>	Airspeed
<code>param7 = 1</code>	ESC

5.6 VEHICLE_CMD_RUN_PREAM_ARM_CHECKS (401)

Triggers an immediate re-evaluation of `_health_and_arwing_checks`. Result visible via `vehicle_status.pre_flight_checks_pass`.

5.7 VEHICLE_CMD_PREFLIGHT_REBOOT_SHUTDOWN (246)

- param1 = 1: warm reboot.
- param1 = 2: full shutdown.

Commander only accepts this when disarmed. Passed to board BSP.

5.8 Other Commands Processed by `handle_command`

Command	Action
NAV_TAKEOFF (22)	Switch to AUTO_TAKEOFF
NAV_LAND (21)	Switch to AUTO_LAND
NAV_RTL (20)	Switch to AUTO_RTL
DO_FLIGHTTERMINATION (185)	Engage kill-switch / lockdown
DO_VTOL_TRANSITION (3000)	Toggle MC↔FW vehicle type
DO_SET_SERVO (183)	Directly actuate a servo (for testing)
FIXED_MAG_CAL_YAW (42006)	Quick yaw magnetometer calibration
SET_GPS_GLOBAL_ORIGIN (100001)	Set EKF NED origin
START_RX_PAIR (500)	Broadcast for GCS pairing

6 `custom_command()` — All CLI Commands

```
1 int Commander::custom_command(int argc, char *argv[])
```

Invoked when the user types `commander <cmd>` in the NuttShell. Returns 0 on success, 1 on error.

6.1 `calibrate`

CLI	Calibration type
<code>commander calibrate gyro</code>	Gyroscope calibration
<code>commander calibrate mag</code>	Magnetometer
<code>commander calibrate mag quick</code>	Quick magnetometer (yaw only)
<code>commander calibrate baro</code>	Barometer
<code>commander calibrate accel</code>	Accelerometer
<code>commander calibrate accel quick</code>	Accelerometer (quick/flat-table)
<code>commander calibrate level</code>	Board level tilt
<code>commander calibrate airspeed</code>	Airspeed sensor offset
<code>commander calibrate esc</code>	ESC PWM range

Each sends `VEHICLE_CMD_PREFLIGHT_CALIBRATION` with the appropriate param populated.

6.2 `check`

Sends `VEHICLE_CMD_RUN_PREARM_CHECKS` and prints whether `pre_flight_checks_pass` is true.

6.3 arm / disarm

Send `VEHICLE_CMD_COMPONENT_ARM_DISARM` with `param1` = ARM or DISARM. With optional `-f` flag: `param2` = 21196 forces the operation.

6.4 takeoff / land

Send `VEHICLE_CMD_NAV_TAKEOFF` / `VEHICLE_CMD_NAV_LAND`. `takeoff` also waits for the ACK reply and then arms automatically.

6.5 transition (VTOL)

Reads current vehicle type and sends `VEHICLE_CMD_DO_VTOL_TRANSITION` to toggle between multi-copter and fixed-wing.

6.6 mode

CLI	DO_SET_MODE payload
<code>mode manual</code>	MAIN=MANUAL
<code>mode altctl</code>	MAIN=ALTCTL
<code>mode posctl</code>	MAIN=POSCTL
<code>mode position:slow</code>	MAIN=POSCTL, SUB=SLOW
<code>mode auto:mission</code>	MAIN=AUTO, SUB=MISSION
<code>mode auto:loiter</code>	MAIN=AUTO, SUB=LOITER
<code>mode auto:rtl</code>	MAIN=AUTO, SUB=RTL
<code>mode auto:takeoff</code>	MAIN=AUTO, SUB=TAKEOFF
<code>mode auto:land</code>	MAIN=AUTO, SUB=LAND
<code>mode auto:precland</code>	MAIN=AUTO, SUB=PRECLAND
<code>mode acro</code>	MAIN=ACRO
<code>mode offboard</code>	MAIN=OFFBOARD
<code>mode stabilized</code>	MAIN=STABILIZED
<code>mode ext1</code>	MAIN=AUTO, SUB=EXTERNAL1

6.7 lockdown

`commander lockdown on`: sends `DO_FLIGHTTERMINATION` with `param1` = 2.0f (lockdown active).

`commander lockdown off`: sends with `param1` = 0.0f.

6.8 pair

Broadcasts `VEHICLE_CMD_START_RX_PAIR` to all systems for GCS pairing.

6.9 set_ekf_origin <lat> <lon> <alt>

Parses three numeric arguments and sends `VEHICLE_CMD_SET_GPS_GLOBAL_ORIGIN` to set the EKF NED reference.

6.10 poweroff

Sends `VEHICLE_CMD_PREFLIGHT_REBOOT_SHUTDOWN` with `param1 = 2.0f` (full power-off).

7 executeActionRequest()

```
1 void Commander::executeActionRequest(const action_request_s &
    action_request)
```

Processes `action_request_s` events, which come from:

- RC mode-switch channels.
- Joystick buttons.
- `ManualControl` module.

Action types handled:

Action	Behaviour
<code>ACTION_ARM</code>	Attempt to arm
<code>ACTION_DISARM</code>	Attempt to disarm
<code>ACTION_TOGGLE_ARMING</code>	Arm if disarmed, disarm if armed
<code>ACTION_KILL</code>	<code>_actuator_armed.manual_lockdown = true</code>
<code>ACTION_UNKILL</code>	Clear manual lockdown
<code>ACTION_SWITCH_MODE</code>	Maps to the requested <code>nav_state</code>
<code>ACTION_VTOL_TRANSITION</code>	Calls the VTOL transition command

8 handleModeIntentionAndFailsafe()

See Section 2.2 step 16 for the step-by-step description. The failsafe priority order (highest to lowest):

Condition	Default action
Flight termination triggered	Terminate
Manual kill switch	Disarm
Battery emergency	Land
Battery critical	Return
RC link loss (in auto)	Continue / RTL
GCS link loss (in auto)	Continue / RTL
Offboard link loss	Hold / RTL
Battery low	Warn
Mission not valid at takeoff	Hold

9 updateControlMode()

```
1 void Commander::updateControlMode()
```

Rebuilds and publishes `vehicle_control_mode` from `nav_state`. Each navigation state maps to a combination of boolean flags:

- `flag_control_manual_enabled`: true in MANUAL, ACRO, STAB.
- `flag_control_rates_enabled`: true in all controlled modes.
- `flag_control_attitude_enabled`: true in STAB and above.
- `flag_control_altitude_enabled`: true in ALTCTL and above.
- `flag_control_position_enabled`: true in POSCTL and AUTO.
- `flag_control_auto_enabled`: true in all AUTO sub-modes.
- `flag_control_offboard_enabled`: true in OFFBOARD.

10 `control_status_leds()`

```
1 void Commander::control_status_leds(bool changed, const uint8_t
    battery_warning)
```

Maps current system state to LED patterns:

Condition	LED colour	Pattern
Disarmed, preflight ok	Green	Slow blink
Disarmed, preflight fail	Red	Fast blink
Armed, GPS ok	Green	Solid
Armed, no GPS	Amber	Solid
Failsafe active	Red	Fast double-blink
Battery critical	Red	Fast blink

11 `dataLinkCheck()`, `manualControlCheck()`, `battery_status_c`

11.1 `dataLinkCheck()`

- Iterates over all `datalink_status` instances (up to 3 links).
- Detects signal loss: `hrt_elapsed_time(last_heartbeat) > DL_LOSS_T`.
- Notifies the failsafe state machine via `_failsafe_flags.link_loss`.

11.2 `manualControlCheck()`

- Reads `manual_control_input`.
- Computes RC loss: elapsed time since last update $> \text{COM_RC_LOSS_T}$.
- Maps stick-arming gesture: both sticks held in arming corner for 2 seconds.
- Sends mode-switch `action_request` when RC switches change.

11.3 battery_status_check()

- Iterates over all battery instances.
- Computes capacity-weighted aggregate battery level.
- Levels: BATTERY_WARNING_NONE / LOW / CRITICAL / EMERGENCY.
- Emergency: notifies failsafe for immediate RTL/land.

12 Other Functions

Function	Purpose
getPrearmState()	True if ESC power-on checks are satisfied
handlePowerButtonState()	Detect long power-button press and shutdown
checkForMissionUpdate()	Detect new mission loaded from GCS
handleAutoDisarm()	Auto-disarm on landing or after timeout
systemPowerUpdate()	USB power, circuit-breakers
checkWorkerThread()	Poll calibration worker for results
updateTunes()	Drive the buzzer for current state (ready, armed, fail)
answer_command()	Publish a <code>vehicle_command_ack</code>
send_parachute_command()	Trigger parachute on kill/termination
printRejectMode()	Log a mode-rejection message via MAVLink
print_status()	Print arm/disarm state to console
print_usage()	Print CLI help

Part II

Soaring-Mode Modifications

13 Overview of Changes

Two changes were made to Commander for soaring:

1. Five new CLI sub-commands under `mode` in `custom_command()`.
2. Modified `VEHICLE_CMD_DO_REPOSITION` handler in `handle_command()` to pass through `param2=0`.

All other Commander functionality is **unchanged**.

14 Modification C1: New CLI Soaring Commands

14.1 What Was Added

Inside `custom_command()`, within the `mode` argument block, five new else-if branches were appended:

```

1 } else if (!strcmp(argv[1], "soar:glide")) {
2     // SOARING_GLIDE_FIXED (mode 1):
3     // engine off, fixed EAS = FW_GLIDE_AIRSPD, follow mission
4     send_vehicle_command(VEHICLE_CMD_CUSTOM_0, 1.0f, 0.0f);
5     send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
6                         PX4_CUSTOM_MAIN_MODE_AUTO,
7                         PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
8
9 } else if (!strcmp(argv[1], "soar:polar")) {
10    // SOARING_GLIDE_POLAR (mode 2):
11    // engine off, polar best-glide EAS = sqrt(B/A), follow mission
12    send_vehicle_command(VEHICLE_CMD_CUSTOM_0, 2.0f, 0.0f);
13    send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
14                        PX4_CUSTOM_MAIN_MODE_AUTO,
15                        PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
16
17 } else if (!strcmp(argv[1], "soar:thermal")) {
18    // SOARING_THERMAL_LOITER (mode 3):
19    // engine off, radius-guided loiter via navigator
20    send_vehicle_command(VEHICLE_CMD_CUSTOM_0, 3.0f, 0.0f);
21    send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
22                        PX4_CUSTOM_MAIN_MODE_AUTO,
23                        PX4_CUSTOM_SUB_MODE_AUTO_LOITER);
24
25 } else if (!strcmp(argv[1], "soar:bank")) {
26    // SOARING_THERMAL_BANK (mode 4):
27    // engine off, direct bank-angle control (NPFG bypassed)
28    send_vehicle_command(VEHICLE_CMD_CUSTOM_0, 4.0f, 0.0f);
29    send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
30                        PX4_CUSTOM_MAIN_MODE_AUTO,
31                        PX4_CUSTOM_SUB_MODE_AUTO_LOITER);
32
33 } else if (!strcmp(argv[1], "soar:off")) {
34    // SOARING_OFF (mode 0): restore powered flight + return to mission
35    send_vehicle_command(VEHICLE_CMD_CUSTOM_0, 0.0f, 0.0f);
36    send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
37                        PX4_CUSTOM_MAIN_MODE_AUTO,
38                        PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
39 }

```

14.2 Two-Command Pattern

Every soaring CLI command sends **two** vehicle commands:

Command	param1	Effect
VEHICLE_CMD_CUSTOM_0	soaring_mode (0–4)	Sets <code>_autosoaring_control.soaring_mode</code> in fw_pos_control
VEHICLE_CMD_DO_SET_MODE	mode flags	Ensures correct nav state in Commander/Navigator

The two-command design is necessary because `CUSTOM_0` is only consumed by `fw_pos_control`, not by Commander itself. Commander therefore needs a separate mode-switch command to set the correct navigation state.

When the operator runs `commander mode soar:off`, `fw_pos_control` also publishes `autosoaring_status` on uORB (ROS2: `/fmu/out/autosoaring_status`) so an XRCE-DDS companion learns the 5s DDS inhibit deadline and can align its `AutosoaringControl` publishes before the FMU accepts DDS re-enable commands again.

14.3 VEHICLE_CMD_CUSTOM_0 Processing (in `fw_pos_control`)

`fw_pos_control` polls `vehicle_command` and handles `VEHICLE_CMD_CUSTOM_0`:

```

1 case vehicle_command_s::VEHICLE_CMD_CUSTOM_0: {
2     const int mode = (int)roundf(cmd.param1);
3     if (mode >= 0 && mode <= 4) {
4         _autosoaring_control.soaring_mode = (uint8_t)mode;
5         _soaring_local_override = true;
6     }
7     break;
8 }

```

The `_soaring_local_override` flag has three effects:

1. Bypass the altitude floor check (`FW_ALT_MIN`) for ground testing at arbitrary altitude.
2. Disable the XRCE-DDS staleness watchdog (no companion expected).
3. Keep the soaring mode active even when the `autosoaring_control` topic is absent.

14.4 Override Clearing Conditions

The override is cleared when:

- `VEHICLE_CMD_CUSTOM_0` with `param1 = 0` is received; or
- `commander mode soar:off` is issued; or
- The aircraft disarms (all state reset).

15 Modification C2: DO_REPOSITION param2=0 Pass-Through

15.1 Original Behaviour

In the original stock code, `handle_command()` for `VEHICLE_CMD_DO_REPOSITION` responded with `VEHICLE_CMD_RESULT_UNSUPPORTED` when `param2 == 0` (“no mode switch requested”).

This blocked the navigator from receiving the coordinate update when the caller did not want a mode transition.

15.2 Modified Behaviour

The new code accepts `param2 == 0`:

```

1 if (mode_switch_not_requested) {
2     // Position-only update: accept without switching mode
3     cmd_result = vehicle_command_ack_s::VEHICLE_CMD_RESULT_ACCEPTED;

```

Why this matters for soaring: When `fw_pos_control` sends `DO_REPOSITION` to move the thermal loiter centre without explicitly requesting a mode switch (the vehicle is already in `LOITER`), `param2 = 0`. Without this fix, Commander would reject the command, and the navigator would never update the loiter centre, making the thermal loiter centre immovable.

15.3 Impact on Non-Soaring Flight

No impact. In normal flight, GCS-generated `DO_REPOSITION` commands always set `param2 = 1` (mode switch requested), which is handled by the existing code path.

16 Summary Table: All Modifications to Commander

Aspect	Original	Modified
CLI soaring commands	None	5 commands (<code>soar:off/glide/polar/thermal/bank</code>)
<code>CUSTOM_0 param1</code>	Undefined	Direct <code>soaring_mode</code> integer (0–4)
<code>DO_REPOSITION param2=0</code>	Rejected (<code>UNSUPPORTED</code>)	Accepted, navigator processes coordinates
Test workflow	Parameter panel + GCS	Single shell command
Override semantics	Not present	<code>_soaring_local_override</code> flag (bypasses floor + watchdog)